# Adaptive Multi-Resolution Grid-Based Clustering for Massive Data Streams

Alireza Zarei
Department of Mathematics and Computer Science
Sharif University of Technology
zarei@sharif.ir
Mohammad Saleh Bahrami
Department of Mathematics and Computer Science
Sharif University of Technology
saleh.bahrami98@sharif.edu

**Abstract**

The rapid growth of streaming data generated by sensors, Internet of Things devices, online platforms, and scientific applications has created a need for clustering algorithms that can process data online under strict memory constraints. Grid-based density clustering methods, such as D-Stream, are attractive for data stream clustering because they summarize incoming points using grid cells instead of storing the entire stream. However, fixed-resolution grid methods suffer from two major limitations: their memory consumption grows rapidly with dimensionality, and a single global cell size may be unable to capture clusters with different local densities or evolving structures.

In this paper, we propose an adaptive multi-resolution grid-based framework for clustering massive data streams. The method maintains a hierarchical grid structure in which each cell stores compact statistical summaries, including time-decayed density, center of mass, variance, and update time. Cells are dynamically refined when their current resolution is insufficient to represent the local data distribution, and they are merged when high resolution is unnecessary or memory pressure increases. Sparse cells are periodically pruned to reduce memory usage. Clusters are extracted from dense connected grid regions using a density-based procedure inspired by DBSCAN.

The proposed framework improves the memory–accuracy trade-off of fixed-resolution grid-based stream clustering algorithms and provides a flexible mechanism for handling non-uniform data distributions and concept drift.

**Keywords:** Data stream clustering; massive data; density-based clustering; grid-based clustering; multi-resolution grid; D-Stream; concept drift; memory-efficient algorithms.

## 1 Introduction

Massive data streams arise naturally in many modern applications, including sensor networks, Internet of Things systems, financial transactions, web logs, network monitoring, and computational biology. In such settings, data points arrive continuously and potentially without bound. Therefore, traditional clustering algorithms, which assume that the entire dataset is stored in memory and can be processed multiple times, are often infeasible.

The data stream model imposes two important constraints. First, the algorithm must process each data point online, usually in one pass. Second, the available memory is significantly smaller than the size of the stream. As a result, stream clustering algorithms typically maintain a compact summary of the observed data rather than storing all incoming points.

Clustering is one of the fundamental tasks in unsupervised learning and data mining. Its goal is to group similar objects together while separating dissimilar objects. Among different clustering approaches, density-based methods are especially useful because they can detect clusters of arbitrary shape and distinguish noise or outliers from meaningful dense regions. However, applying density-based clustering directly to data streams is challenging because density estimates must be updated continuously and efficiently.

Grid-based density clustering methods address this challenge by partitioning the data space into cells and maintaining density summaries for these cells. A well-known example is D-Stream, which uses a fixed-resolution grid and a fading function to give more importance to recent data. Although such methods are efficient, they suffer from important limitations. The number of cells grows exponentially with the dimension of the data, and a fixed grid resolution may be inappropriate for data whose density varies across space or changes over time.

To address these limitations, this paper proposes an adaptive multi-resolution grid-based clustering framework for massive data streams. Instead of using a single global grid resolution, the proposed method dynamically refines or merges grid cells according to local statistical properties of the stream and the available memory budget. Dense or complex regions can be represented by smaller cells, while sparse or simple regions can be summarized by larger cells.

The main idea is to maintain a hierarchical grid structure, such as a quadtree in two dimensions or a $2^d$-tree in $d$ dimensions. Each cell stores a compact feature vector describing the local data distribution. Based on this vector, the algorithm decides whether the cell should be refined, merged, pruned, or kept unchanged.

### 1.1 Contributions

The main contributions of this paper are as follows:

- We propose an adaptive multi-resolution grid framework for density-based clustering of massive data streams.
- We introduce dynamic refinement and contraction operations that allow the grid resolution to change over both space and time.
- We maintain compact cell-level summaries, including time-decayed density, center of mass, variance, and update time, to guide refinement and merging decisions.
- We incorporate sparse-cell pruning to reduce memory consumption and remove cells that have negligible influence on clustering quality.
- We describe a density-based clustering procedure over active grid cells, reducing the need to cluster raw data points.
- We provide a framework that can be viewed as an adaptive multi-resolution extension of fixed-grid stream clustering algorithms such as D-Stream.

## 2 Related Work

Data stream clustering has been widely studied due to the increasing importance of real-time data analysis. Existing methods can be broadly categorized into micro-cluster-based methods, density-based methods, grid-based methods, and hybrid approaches.

CluStream is one of the early frameworks for clustering data streams. It separates the process into an online micro-clustering phase and an offline macro-clustering phase. DenStream extends the micro-cluster idea using density-based potential and outlier micro-clusters. DBSTREAM also follows a density-based strategy and maintains shared density information between micro-clusters.

Grid-based methods partition the data space into cells and maintain summaries for these cells. STING is an early statistical grid-based method that stores statistical information in hierarchical grid cells. D-Stream adapts grid-based clustering to data streams by using a density decay factor and periodically removing sparse grids. These methods are efficient, but their performance depends heavily on the choice of grid resolution.

The main weakness of fixed-grid approaches is that one global cell size cannot represent all regions of a non-uniform or evolving data stream equally well. If the cell size is too large, distinct clusters may be merged. If the cell size is too small, the number of cells may become too large, causing high memory consumption and computational overhead.

**Multi-resolution and adaptive-grid approaches.** The idea of letting resolution vary across a stream rather than fixing it once is not new. HASTREAM [9] builds a hierarchical density-based model over micro-clusters and adapts its density threshold independently per region without user intervention, closest in spirit to the present work's goal, though its mechanism is a hierarchy over micro-clusters rather than a split/merge grid, and it does not maintain the explicit per-cell moment statistics this paper's refinement criterion is built on. Earlier adaptive-grid proposals include mesh-refinement-style grids that apply different granularities to regions of different density, and accuracy-guaranteed adaptive-grid $k$-median clustering; both predate the time-decayed density model and the split/merge/prune loop used here, and neither reports the memory-accounting or dense-threshold interactions identified in Section 8.3. To our knowledge, no prior grid-based method couples split/merge decisions directly to a per-cell equidistribution test of the kind studied in Section 8.6 below.

**Non-convex and arbitrary-shape streaming clusters.** CEDAS [7] and, more recently, MCMSTStream [13] address arbitrary cluster shape directly, the latter by running a minimum spanning tree over KD-tree-indexed micro-clusters rather than axis-aligned cells. This is a relevant contrast to the present work's own moons-drift result (Section 8.7), where axis-aligned rectangular cells and face-adjacency connectivity are shown empirically to be a poor fit for non-convex boundaries at any grid resolution; combining a micro-cluster-style shape mechanism with the memory accounting of a grid is a natural extension, noted in Future Work.

**Concept drift beyond time-decay.** This paper follows D-Stream's exponential fading for drift adaptation. An alternative recent direction augments grid-density clustering with explicit drift detectors: Rezaei and Sajedi [14] combine a grid-density-based clustering model with dual drift-detection mechanisms based on distance metrics and fractal dimension, triggering re-partitioning rather than relying on decay alone. Explicit drift detection of this kind is listed as a future-work direction in this paper as well (see Future Work below), independent of, and potentially complementary to, the density-normalization fix identified as the current bottleneck in Section 8.3.

**Where grid-based methods stand in recent surveys.** Broader stream-clustering surveys and evaluations, from Mansalis et al. [10] and Ghesmoune et al. [11] through Tareq et al.'s systematic review specifically of density-grid-based methods [12] and the most recent evaluation survey by Nordahl et al. [15], consistently report that density-grid-based algorithms trail micro-cluster-based methods (DenStream-, DBSTREAM-, and CluStream-style approaches) on cluster quality for evolving streams, and identify this as an open, recurring issue across the density-grid literature rather than a property of any single implementation. This is consistent with, rather than contradicted by, the empirical ordering found throughout Section 8 of this paper (imported `river` baselines outperforming both the fixed-grid frontier and `AdaptiveDStream` on ARI at a fraction of the memory): it situates the gap identified in Section 8.3 as an instance of

a wider, already-documented pattern in the density-grid family, not an anomaly specific to this prototype.

Our method is motivated by these limitations. It combines the compact representation of grid-based methods, the time-decayed density mechanism of D-Stream, the statistical summaries of STING-like methods, and the sparse-cell pruning philosophy of stream clustering algorithms such as CEDAS.

# 3 Problem Definition

Let

$$X = \{x_1, x_2, \ldots, x_t, \ldots\}$$

be a data stream, where each point $x_t \in \mathbb{R}^d$ arrives at time $t$. The stream may be very large or potentially infinite, so the algorithm cannot store all points.

The goal is to maintain, at any time $t$, a clustering of the points observed so far, or more precisely, a clustering of their compact summaries. The algorithm should satisfy the following requirements:

- It should process each point online.
- It should use memory significantly smaller than the number of observed points.
- It should adapt to changes in the data distribution.
- It should detect clusters of arbitrary shape.
- It should handle noise and sparse regions efficiently.

## 3.1 Time-Decayed Density

Following D-Stream, we assign higher importance to recent data points. Let $\lambda \in (0, 1)$ be a decay factor. If a point $x_i$ arrived at time $T(x_i)$, its density contribution at time $t$ is defined as

$$D(x_i, t) = \lambda^{t-T(x_i)}.$$

For a grid cell $g$, the time-decayed density at time $t$ is

$$D(g, t) = \sum_{x_i \in E(g,t)} \lambda^{t-T(x_i)},$$

where $E(g, t)$ is the set of points assigned to cell $g$ before or at time $t$.

If the last update time of cell $g$ is $t_l$ and a new point arrives at time $t_n$, the density can be updated efficiently by

$$D(g, t_n) = \lambda^{t_n - t_l} D(g, t_l) + 1.$$

This update requires constant time for the affected cell.

# 4 Proposed Method

The proposed method maintains an adaptive hierarchical grid. Unlike fixed-resolution grid methods, different regions of the data space may be represented at different resolutions.

### 4.1 Hierarchical Grid Structure

The data space is initially represented by a coarse grid. Each grid cell may be recursively divided into smaller child cells. In two dimensions, this structure corresponds to a quadtree. In $d$ dimensions, each refinement divides a cell into $2^d$ child cells.

Each cell $g$ has a level $\ell(g)$, where lower levels correspond to coarser cells and higher levels correspond to finer cells. The side length of a cell at level $\ell$ is denoted by $h_\ell$.

### 4.2 Cell Summary

Each active cell stores a compact summary vector:

$$\Phi(g) = \big(D(g), \mu(g), \sigma^2(g), t_g, \ell(g), n(g)\big)\,,$$

where:

- $D(g)$ is the time-decayed density of the cell,
- $\mu(g)$ is the time-decayed center of mass,
- $\sigma^2(g)$ is the estimated variance,
- $t_g$ is the last update time,
- $\ell(g)$ is the cell level,
- $n(g)$ is the number of points or weighted count assigned to the cell.

The center of mass can be updated online. If a new point $x_t$ is assigned to cell $g$, then after applying decay to the previous statistics, the updated center can be computed as

$$\mu(g,t) = \frac{\lambda^{t-t_g} D(g,t_g)\mu(g,t_g) + x_t}{\lambda^{t-t_g} D(g,t_g) + 1}.$$

Similar online updates can be used for the variance.

### 4.3 Refinement Criterion

A cell should be refined when its current resolution is not sufficient to represent the local data distribution. We define a refinement score:

$$R(g) = \alpha_1 \cdot \frac{\sigma^2(g)}{h^2_{\ell(g)}} + \alpha_2 \cdot \frac{\|\mu(g) - c(g)\|^2}{h^2_{\ell(g)}} + \alpha_3 \cdot \rho(g),$$

where:

- $c(g)$ is the geometric center of the cell,
- $h_{\ell(g)}$ is the side length of the cell,
- $\rho(g)$ is an optional local density irregularity measure,
- $\alpha_1, \alpha_2, \alpha_3$ are weighting parameters.

The intuition is as follows. If the variance inside a cell is large, or if the center of mass is far from the geometric center, then the assumption that data is uniformly distributed inside the cell may be inaccurate. In such cases, the cell may need to be refined.

A cell $g$ is refined if

$$R(g) > \tau_{\text{split}},$$

provided that the memory budget allows refinement and the maximum grid depth has not been reached.

### 4.4 Alternative Refinement Criteria: Weyl and Random-Projection Tests

$R(g)$ is a moment-matching statistic: it compares the cell's tracked mean and variance to the values they would take under a uniform distribution, and is blind, by construction, to any alternative that preserves those two moments while changing the shape of the distribution (e.g. points concentrated along a diagonal of the cell, or a symmetric bimodal split). This subsection introduces two criteria that test equidistribution more directly, both compatible with the same $O(1)$-per-point streaming update discipline as $R(g)$, and both with a threshold $\tau$ that is derived from an explicit probabilistic guarantee rather than tuned as a free constant. Throughout, let $u = (x - \ell(g))/h_{\ell(g)} \in [0,1]^d$ denote a point rescaled into the cell's own local unit cube, so that $H_0$ ("$g$ is locally equidistributed") is exactly $u \sim \text{Uniform}([0,1]^d)$.

#### 4.4.1 The Weyl Criterion

For a nonzero integer vector $k \in \mathbb{Z}^d \setminus \{0\}$, define the character $\varphi_k(u) = \exp(2\pi i\, k \cdot u)$ and its sample mean over the $n$ points currently in $g$, $S_k = \frac{1}{n}\sum_{j=1}^{n} \varphi_k(u_j)$. Because the integer characters form an orthonormal basis of $L^2([0,1]^d)$, under $H_0$ each $\varphi_k$ has mean 0 and unit modulus exactly, so $\text{Var}(\text{Re}\,\varphi_k) = \text{Var}(\text{Im}\,\varphi_k) = \frac{1}{2}$ with zero covariance, and any two distinct frequencies $k \neq \pm k'$ are exactly uncorrelated (Proposition 1, Appendix A). Fix a frequency set $\mathcal{K} = \{k_1, \ldots, k_m\} \subset \mathbb{Z}^d \setminus \{0\}$ with no two frequencies equal or opposite — we use the $d$ axis unit vectors together with the sum and difference of each consecutive coordinate pair, giving $m = 3d - 2$, linear in $d$ — and define the Weyl statistic

$$T_{\text{Weyl}}(g) = \sum_{k \in \mathcal{K}} 2n\, |S_k|^2.$$

By the multivariate central limit theorem, $T_{\text{Weyl}}(g) \Rightarrow \chi^2_{2m}$ under $H_0$ as $n \to \infty$ (Theorem 1, Appendix A). The refinement threshold is therefore not a tuned constant but the $(1-\alpha)$ quantile of $\chi^2_{2m}$ for a chosen significance level $\alpha$: $g$ is refined if $T_{\text{Weyl}}(g) > \chi^2_{2m}(1-\alpha)$. Unlike $R(g)$, which only aggregates per-axis statistics, $\mathcal{K}$'s sum/difference frequencies give $T_{\text{Weyl}}$ sensitivity to cross-axis (e.g. diagonal) concentration; Section 8.6 measures this directly. The cost is $O(nm) = O(nd)$ per evaluation, versus $O(n \cdot 2^d)$ for a literal $2^d$-orthant $\chi^2$ test. A finite frequency set has blind directions — alternatives whose departure from uniformity lies entirely in an untested harmonic — which is a known property of Fourier-type goodness-of-fit tests in the tradition of Neyman's smooth test [17]; Section 8.6 exhibits one such case.

#### 4.4.2 The Random-Projection Criterion

An alternative to testing specific harmonics of $u$ is to test linear projections of $u$ directly, in the spirit of random-projection goodness-of-fit testing [18]. Fix a unit vector $v \in \mathbb{R}^d$ ($\|v\|_2 = 1$) and let $Y = v \cdot u$. Under $H_0$, $Y$ is a sum of independent scaled uniforms, so $\mathbb{E}[Y] = \frac{1}{2}\sum_j v_j$ and $\text{Var}(Y) = \frac{1}{12}\|v\|_2^2 = \frac{1}{12}$ exactly, independent of $v$ and $d$, and $Y$ is confined to an interval of width $R(v) = \sum_j |v_j|$ (Proposition 2, Appendix A). $Y$'s distribution has no simple closed form for general $v$ (it is a weighted, generalized Irwin–Hall law), so rather than an asymptotic approximation we use a finite-sample concentration bound. Fix $m$ random unit directions $v_1, \ldots, v_m$, drawn once and independent of the data. For each $v_i$ we run two elementary tests, both comparing a sample mean of i.i.d. bounded variables to its exact null value via Hoeffding's inequality [16]: a *mean* sub-test on $Y = v_i \cdot u$ against $\frac{1}{2}\sum_j v_{i,j}$, and a *variance* sub-test on $Z = (Y - \frac{1}{2}\sum_j v_{i,j})^2 \in [0, R(v_i)^2]$ against $\frac{1}{12}$. Running each of the resulting $2m$ elementary tests at level $\alpha/(2m)$ and refining $g$ if any one rejects gives, by a union bound,

$$\Pr(\text{refine } g \mid H_0) \;\leq\; \alpha \qquad \text{for every finite } n$$

(Corollary 1, Appendix A). This is a strictly stronger guarantee, in one respect, than the Weyl criterion's: it holds at every $n$, not only asymptotically. The price is that Hoeffding's inequality uses only boundedness, not the true variance, so it is loose; and since a random unit vector in $\mathbb{R}^d$ has $\mathbb{E}[\,|v_j|\,] = \Theta(1/\sqrt{d})$ per coordinate, its range grows as $R(v) = \Theta(\sqrt{d})$, appearing squared in the bound's exponent. A fixed true deviation therefore becomes exponentially harder to certify, at fixed $n$, as $d$ grows — a concrete cost of the non-asymptotic guarantee, measured in Section 8.6. As with the Weyl criterion, $m$ directions cost $O(nm) = O(nd)$ per evaluation for $m = O(d)$, and are drawn once (fixed by a seed derived from $d$) so the criterion is deterministic and reproducible.

### 4.5 Contraction Criterion

The algorithm may merge child cells into their parent when high resolution is unnecessary. Let $g_1, \ldots, g_{2^d}$ be the children of a parent cell $p$. These children can be merged if their combined representation is sufficiently close to the parent-level approximation.

A possible contraction condition is

$$\max_i R(g_i) < \tau_{\text{merge}},$$

or

$$\Delta(p) < \epsilon_{\text{merge}},$$

where $\Delta(p)$ measures the information loss caused by replacing the children with their parent.

Contraction is especially useful when memory usage approaches the allowed memory budget.

### 4.6 Sparse Cell Pruning

Cells with very low decayed density are unlikely to contribute significantly to the final clustering. Therefore, the algorithm periodically removes sparse cells.

A cell $g$ is pruned if

$$D(g,t) < \tau_{\text{sparse}},$$

and it has not received new points for a sufficiently long time. "Not received new points" is measured from a per-cell last-arrival timestamp that only a point landing in the cell updates; an earlier implementation measured it from the last decay update, which every clustering pass resets, so pruning silently never fired. A point that later falls in a pruned region is assigned to the nearest surviving sibling cell.

This pruning mechanism reduces memory usage and allows the algorithm to focus on active dense regions of the stream.

### 4.7 Clustering over Grid Cells

At query time or after a fixed number of updates, the algorithm extracts clusters from the active grid cells.

Each active cell is labeled as:

- **dense**, if $D(g,t) \geq \tau_{\text{dense}}$,
- **transitional**, if $\tau_{\text{sparse}} \leq D(g,t) < \tau_{\text{dense}}$,
- **sparse**, if $D(g,t) < \tau_{\text{sparse}}$.

A graph is constructed over dense cells. Two cells are connected if they are spatial neighbors and their resolutions are compatible. Clusters are then obtained as connected components of this

**Algorithm 1** Adaptive Multi-Resolution Grid Stream Clustering

Initialize root grid cells.
Set decay factor $\lambda$, thresholds $\tau_{\text{split}}$, $\tau_{\text{merge}}$, $\tau_{\text{sparse}}$, and memory budget $M$.
**for** each incoming point $x_t$ **do**
  Locate the finest active cell $g$ containing $x_t$.
  Apply time decay to the statistics of $g$.
  Update $D(g)$, $\mu(g)$, $\sigma^2(g)$, and $t_g$.
  **if** $R(g) > \tau_{\text{split}}$ and memory usage is below $M$ **then**
    Refine $g$ into $2^d$ child cells.
    Distribute the summary of $g$ among its children using the available cell statistics.
  **end if**
  **if** memory usage is close to $M$ **then**
    Increase refinement strictness.
    Attempt contraction of low-information child cells.
  **end if**
  **if** pruning condition is triggered **then**
    Remove cells with $D(g, t) < \tau_{\text{sparse}}$.
  **end if**
**end for**
Extract clusters from connected dense cells.

graph. Transitional cells may be assigned to nearby dense clusters if they satisfy a neighborhood condition.

This procedure is similar in spirit to DBSCAN, but it operates on grid cells rather than raw data points.

# 5 Algorithm

# 6 Complexity Analysis

Let $m$ be the number of active cells currently stored by the algorithm. Since the algorithm does not store all observed points, its memory complexity is

$$O(m),$$

where $m$ is controlled by the memory budget and pruning mechanism.

For each incoming point, the algorithm must locate the corresponding active cell and update its summary. If the hierarchical grid has maximum depth $L$, then point insertion requires

$$O(L)$$

time in the worst case. For fixed maximum depth, this becomes constant or logarithmic in the number of active cells depending on the implementation.

The refinement of a cell creates $2^d$ children. Therefore, the cost of a refinement operation is

$$O(2^d).$$

However, refinement is not performed for every point and is controlled by the refinement threshold and memory budget.

The clustering phase operates over active dense cells. If $m_d$ is the number of dense cells, then constructing the adjacency graph and finding connected components requires approximately

$$O(m_d \cdot \eta),$$

where $\eta$ is the number of neighboring cells checked for each cell. In fixed dimension, $\eta$ is usually bounded by a constant.

# 7 Experimental Evaluation

To evaluate the proposed method, it should be compared against representative stream clustering algorithms, including D-Stream, DenStream, CluStream, DBSTREAM, and CEDAS.

## 7.1 Datasets

All evaluation in this paper uses synthetic streams, generated so that ground-truth cluster labels are known at every time step and specific structural properties (density mismatch, drift, shape) can be isolated one at a time. Five regimes were used, all two-dimensional, $n$=2000 points, seed 7:

- **static_no_drift**: a tight, dense Gaussian cluster and a broad, sparse Gaussian cluster, present simultaneously, with both centers held fixed for the entire stream. The no-drift control.
- **spatial_drift**: the same tight/broad pair, but both centers now drift across three phases. This is the adversarial case motivating the method: a cell size small enough to resolve the tight cluster is wasteful over the broad cluster's footprint, while a cell size large enough for the broad cluster swallows the tight cluster whole.
- **density_drift**: centers held fixed (as in **static_no_drift**), but the dense cluster's standard deviation widens phase by phase ($0.15 \rightarrow 0.15 \rightarrow 1.1$). Isolates a change in local density from any change in position.
- **two_gaussian_drift**: two Gaussian clusters of *equal*, constant spread, drifting across three phases. A non-adversarial control: drift without a density mismatch.
- **moons_drift**: two interleaving, non-convex "moon" clusters, rotating and drifting across three phases. Tests arbitrary cluster shape rather than density or drift.

The tight/broad Gaussian pair and its `drift`/density-schedule parameters are implemented in `make_varying_density_stream`; the symmetric and moon-shaped streams are separate generators. All are pure functions of a random seed, so every run is exactly reproducible.

Section 8.8 extends this with three real streaming-clustering benchmarks, the standard triple in the literature: KDD Cup 99 (network intrusion) [19], Forest CoverType (cartographic) [20], and Statlog Shuttle telemetry [21], used to check whether the synthetic-stream findings above generalize to real, correlated feature spaces rather than only isotropic noise dimensions.

## 7.2 Evaluation Metrics

Every run below reports, via a prequential (test-then-train) protocol — each point is scored by the model's current state *before* that model is allowed to update on it:

- Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI),
- clustering purity,

- peak memory, measured from model construction through the full run (so a fixed grid's upfront cell allocation is counted, not only its online updates),
- throughput (points/second) and fraction of points left unassigned.

Grid/density-based models return no label for a point that falls in a non-dense cell. This is scored, not discarded: every such prediction is mapped to a sentinel label treated as its own predicted cluster, following the standard DBSCAN/DenStream "noise" convention — a model that dumps everything into "unassigned" is penalized as if it put all of that into one large incorrect cluster, rather than being scored only on the points it chose to commit to.

The metrics above weight every point in the stream equally regardless of arrival time. That is not a neutral default: `AdaptiveDStream`, `FixedGridDStream`, and two of the three `river` baselines (DenStream's `decaying_factor`, DBSTREAM's `fading_factor` — only CluStream's windowed snapshots genuinely do not decay) all maintain state that deliberately forgets old data, at three different, uncontrolled rates. A flat aggregate ARI/NMI reflects none of that. We therefore additionally report, for a fixed externally-chosen decay rate $\lambda_{\text{eval}}$ applied identically to every model in a sweep (not read from any individual model's own internal decay parameter, since those are not on a comparable scale to begin with): a decay-weighted purity, in which point $i$ (arrival order) is weighted $\lambda_{\text{eval}}^{n-1-i}$, exactly the rate `GridCell.decay_to` uses internally; and a windowed ARI/NMI, ordinary unmodified `adjusted_rand_score`/`normalized_mutual_info_score` recomputed over only the most recent $\lceil 1/(1-\lambda_{\text{eval}}) \rceil$ points, the effective sample size of that decay rate. ARI's chance-correction assumes a specific null distribution over contingency tables that changes under point weights, so rather than risk a subtly incorrect decay-weighted ARI, the windowed version uses trusted, unmodified ARI/NMI math restricted to a recency-focused slice of the stream. Every sweep below uses $\lambda_{\text{eval}} = 0.99$, `AdaptiveDStream`'s own `decay`; Section 8.9 reports what this changes.

Numerical results, including a comparison across the five regimes introduced above, are reported in Section 8.

# 8 Results

This section reports results from a research prototype of the proposed method, implemented in Python, and compared against a swept fixed-resolution D-Stream baseline and three imported reference methods. Since the previous version of this evaluation, three items proposed but not yet implemented in Section 4 have been added: contraction (children merge back into their parent when none of them individually still need the resolution), a border-assignment rule that attaches transitional cells to an adjacent dense cluster rather than leaving them permanently unlabeled, and two alternative split-mass-redistribution strategies alongside the original. We report the resulting numbers as an honest snapshot of an early-stage implementation rather than a demonstration of a finished method: the prototype is still dominated by the baselines on accuracy, though the new features measurably fixed the two things they targeted.

## 8.1 Experimental Setup

We use a synthetic two-dimensional stream, `varying_density` (seed 7, $n = 2000$ points[1]), constructed so that a single fixed grid resolution provably cannot represent both clusters well:

[1]Reduced from $n = 4000$ in the previous version of this evaluation: at the time, `predict_point_cluster` re-ran the $O(\text{dense}^2)$ clustering-adjacency check on every point, and the harness's `tracemalloc` instrumentation multiplied that further, so $n = 4000$ stopped being practical to reproduce. Clustering now runs only at maintenance checkpoints and the sweep takes seconds, but $n = 2000$ was kept so results remain comparable; a full-scale $n = 4000$ re-run is now cheap and remains the more rigorous version of this comparison. The qualitative pattern below is corroborated independently by the regime sweep (Section 8.7) at its own, separately-reduced sample size.

a tight, dense Gaussian cluster ($\sigma = 0.18$) and a broad, sparse Gaussian cluster ($\sigma = 1.6$) are present simultaneously throughout the stream, with both centers drifting across three phases. Models are evaluated prequentially, that is, each incoming point is scored against the model's current state before the model is updated on it.

The reference frontier is a fixed-resolution D-Stream swept over ten grid resolutions, $n_{\text{cells per dim}} \in \{2, 3, 4, 6, 8, 11, 16, 22, 32, 45\}$, tracing out an accuracy-versus-memory curve rather than a single operating point. We additionally import three published stream clustering methods from the `river` library, DenStream, CluStream, and DBSTREAM, without modification, so that the adaptive method is compared against real published baselines and not only against a self-authored fixed-grid ablation. Peak memory is measured with `tracemalloc` from before model construction, so that a fixed grid's upfront cell allocation is counted rather than only the cost of the online update phase. Points that fall in a non-dense cell with no dense neighbor are assigned a sentinel label rather than being dropped, so that a model which abstains on much of the stream is not rewarded for doing so. Unless noted otherwise, `AdaptiveDStream` below uses contraction ($\tau_{\text{merge}} = 0.01$, merge-eligible only once a split is at least 200 points old) and the default `equal_uniform` split strategy; Section 8.4 compares all three strategies directly.

## 8.2 Accuracy–Memory Frontier in Two Dimensions

Table 1 reports peak memory, Adjusted Rand Index (ARI), Normalized Mutual Information (NMI), and the fraction of unassigned points for the fixed-grid frontier, the three imported baselines, and the adaptive method. Figure 1 plots ARI and NMI against peak memory for all models.

**Two changes to the implementation since the previous version of these numbers, and how to read them.** (1) Idle pruning now actually prunes: a bug meant `idle_prune_after` never triggered (the idle test compared against a timestamp that every decay update had just reset), so the adaptive method never released idle leaves; it now tracks the last time a point landed in each cell separately. (2) Clustering now runs at checkpoints, every `maintenance_interval`= 50 points, rather than on every prediction. Change (2) has a cold-start cost shared by every grid model: the first 50 predictions are always unassigned, so fraction unassigned and ARI are depressed by an amount that is large on the 500-point sweeps ($\geq 0.10$ of unassigned mass by construction) and small on the 2000-point stream. Differences from earlier tables are the sum of both changes and were not separated experimentally.

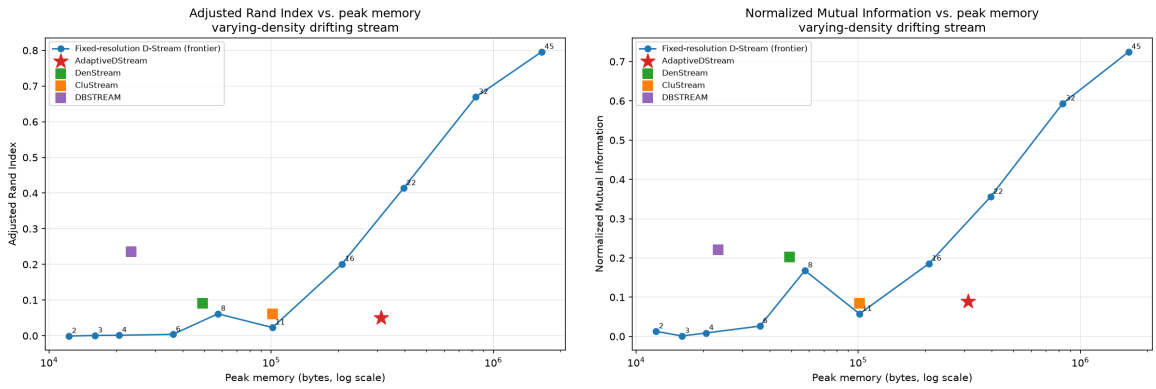



Figure 1: ARI (left) and NMI (right) versus peak memory for the fixed-grid frontier, the three imported baselines, and AdaptiveDStream.

With idle pruning active, `AdaptiveDStream`'s peak memory falls from 666 KB to 304 KB (from 1344 KB in the original $n = 4000$ run), placing it between FixedGrid $n$=16 (203 KB, ARI 0.201) and $n$=22 (385 KB, ARI 0.414). Its own ARI is 0.050 (previously 0.066), below both

Table 1: Accuracy–memory frontier on the varying-density stream (2D, $n = 2000$, seed 7). AdaptiveDStream is highlighted; it is dominated on ARI by the fixed-grid frontier at comparable memory and by DBSTREAM. Clustering runs at checkpoints (every 50 points) and idle pruning is active.

| Model | Peak memory (KB) | ARI | NMI | Frac. unassigned |
|---|---|---|---|---|
| FixedGrid $n$=2 | 12.0 | -0.001 | 0.013 | 0.03 |
| FixedGrid $n$=3 | 15.7 | 0.001 | 0.001 | 0.03 |
| FixedGrid $n$=4 | 20.2 | 0.001 | 0.009 | 0.04 |
| DBSTREAM | 22.8 | 0.236 | 0.221 | 0.00 |
| FixedGrid $n$=6 | 35.3 | 0.004 | 0.026 | 0.05 |
| DenStream | 47.8 | 0.092 | 0.202 | 0.00 |
| FixedGrid $n$=8 | 56.2 | 0.062 | 0.168 | 0.07 |
| FixedGrid $n$=11 | 99.0 | 0.023 | 0.058 | 0.12 |
| CluStream | 99.0 | 0.061 | 0.084 | 0.00 |
| FixedGrid $n$=16 | 203.2 | 0.201 | 0.185 | 0.29 |
| **AdaptiveDStream** | **304.3** | **0.050** | **0.090** | **0.19** |
| FixedGrid $n$=22 | 385.1 | 0.414 | 0.356 | 0.44 |
| FixedGrid $n$=32 | 814.1 | 0.669 | 0.594 | 0.50 |
| FixedGrid $n$=45 | 1609.1 | 0.796 | 0.725 | 0.53 |

neighbors and far below FixedGrid $n$=32 and $n$=45 (0.669 and 0.796); DBSTREAM reaches 0.236 with roughly 13 times less memory (23 KB). Fraction unassigned is 0.19 (previously 0.21, and 0.42 before transitional-cell assignment). The frontier conclusion is therefore unchanged: at every memory budget a fixed grid or an imported baseline is at least as accurate. The same pattern, memory and unassigned-fraction gains without a corresponding ARI gain, recurs in the regime sweep (Section 8.7).

### 8.3 Root-Cause Analysis of the Accuracy Gap

We traced the original gap to two specific, verified implementation issues, both concerning how the dense/sparse rule and memory accounting interact with splitting, rather than the splitting heuristic (when to split) itself. Contraction and transitional assignment (Section 4) were built to address the second; neither touches the first, and the frontier result above shows exactly that split: the issue they target is measurably better, and ARI is not.

1. **The dense threshold is an absolute decayed count, not a density — unchanged by this iteration.** Splitting a cell fragments its accumulated mass across its $2^d$ children. A region can be genuinely dense, that is, have high mass per unit volume, while every fine cell covering it holds too little raw decayed count to cross $\tau_{\text{dense}}$, simply because the same mass is now divided among more cells. Consequently no single absolute threshold value is correct both before and after a region is refined. We originally verified this is the dominant effect by manually sweeping the threshold pair from $(\tau_{\text{dense}}, \tau_{\text{sparse}}) = (2.0, 0.3)$, the value shared with the fixed-grid baseline, down to $(0.5, 0.05)$: ARI improved from 0.040 to only about 0.106 before plateauing, never approaching the fixed-grid frontier. None of contraction, transitional assignment, or the choice of split strategy (Section 8.4) changes how $\tau_{\text{dense}}$ and $\tau_{\text{sparse}}$ are compared against $D(g, t)$, so this remains the top open item.

2. **Splitting still does not fully free the parent cell's storage — contraction now reclaims some of it, quantifiably.** The split operation attaches $2^d$ children to a cell but leaves the parent object itself resident in memory, still holding its sufficient statistics,

unless the children later satisfy the contraction criterion and are merged back. A direct, instrumented comparison on the same stream and configuration as Table 1 (measured before the idle-pruning fix described above, and not repeated since): *without* contraction, the run ends with 754 leaf cells and 1005 total live cell objects (251 retired parents, $\approx 25\%$ dead weight, matching the original finding almost exactly); *with* contraction enabled, it ends with 637 leaf cells and 849 total objects (212 retired parents), a real $\approx 15\%$ reduction in total live cells. Contraction does not eliminate the issue, though: a cell whose children keep legitimately deviating from the locally-uniform assumption, that is, a real, persistent cluster, never satisfies the merge criterion, and its stale parent remains resident indefinitely, exactly as before.

Both issues point toward the same fix as before: making the dense/sparse classification resolution-aware. Memory accounting is now partially, though not fully, addressed by contraction.

### 8.4 Split-Strategy Comparison

A split has to infer how to redistribute a parent's decayed mass across its $2^d$ children from summary statistics alone, since raw observations are not kept. Three strategies are implemented (Section 4): `equal_uniform` (the original default: mass split equally, each child assumed locally uniform), `point_mass` (a degenerate baseline: all mass to whichever single child contains the parent's tracked mean), and `moment_based` (an axis-independent Gaussian model of the parent, truncated to its bounds, giving each child the Gaussian mass and truncated-normal moments its half-interval implies). Table 2 compares all three, with contraction enabled, on the same adversarial stream (`drift=true`, $n = 800$, seed 7), holding every other hyperparameter fixed.

Table 2: Split-strategy comparison, varying-density drifting stream ($n = 800$, seed 7). Only `split_strategy` differs between the three AdaptiveDStream rows.

| Model | Peak memory (KB) | ARI | NMI | Frac. unassigned |
|---|---|---|---|---|
| DBSTREAM | 21.8 | 0.019 | 0.048 | 0.00 |
| FixedGrid $n$=8 | 59.1 | 0.043 | 0.111 | 0.12 |
| DenStream | 40.2 | 0.062 | 0.155 | 0.00 |
| CluStream | 98.1 | 0.112 | 0.118 | 0.00 |
| **AdaptiveDStream** (`equal_uniform`) | 258.7 | **0.033** | 0.090 | 0.12 |
| **AdaptiveDStream** (`moment_based`) | 326.0 | **0.033** | 0.081 | 0.16 |
| **AdaptiveDStream** (`point_mass`) | 356.6 | **0.020** | 0.066 | 0.38 |
| FixedGrid $n$=32 | 811.0 | 0.504 | 0.474 | 0.58 |

`equal_uniform` and `moment_based` now tie on ARI (0.033 each), and `point_mass` is clearly the worst, for an explainable reason. (Previously `equal_uniform` was narrowly ahead, 0.024 against 0.021, a difference that did not survive the implementation changes above and was within noise at this sample size.) `equal_uniform` remains the cheaper of the two (259 KB vs. 326 KB) with lower fraction unassigned (0.12 vs. 0.16). `point_mass` hands each split's entire mass to one child and leaves the other $2^d - 1$ at exactly zero, so those children stay `sparse` until real new data arrives, inflating fraction unassigned to 0.38 (2–3× the other two strategies) while also using the most memory of the three, since an empty child is still a live `GridCell` object. `moment_based` spreads mass more realistically than `point_mass` (verified separately by an exact mass-conservation test), but that does not translate into a measurable ARI gain over the even split on this stream. None of the three strategies brings `AdaptiveDStream` close to the fixed-grid or `river` baselines: this experiment isolates the split-mass-redistribution question

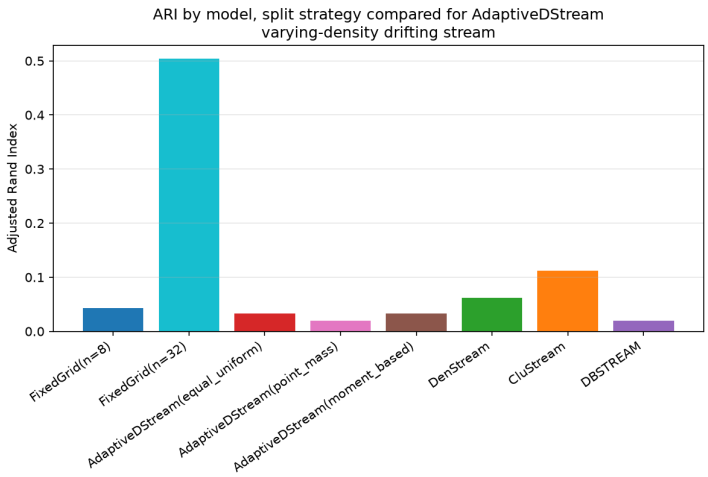



Figure 2: ARI by model on the split-strategy comparison of Table 2.

specifically, and the answer is that it is not where the current ARI gap originates, consistent with Section 8.3.

## 8.5 Effect of Dimensionality

We repeated the comparison on the same stream family generalized to $d > 2$, with the two cluster centers orbiting in the first two coordinates and every additional dimension carrying pure isotropic noise, the standard curse-of-dimensionality stress case. This is a preliminary probe rather than a full multi-dimensional evaluation: it uses a single seed, $n = 400$ points (reduced from 2500 for the same reason as Section 8's experimental setup, and more acutely here since a split costs $O(2^{\text{dim}})$ regardless of strategy and contraction adds a second full-tree pass every maintenance interval), and neither model's thresholds were retuned per dimension. At each dimension, the fixed grid's resolution was chosen so its total cell count lands near the adaptive method's 2500-cell budget, so the comparison is roughly memory-matched rather than resolution-matched; past $d = 15$ the fixed grid's own resolution floor of $n_{\text{cells per dim}} = 2$ pushes its cell count itself past $2^{20} \approx 10^6$, at which point it is skipped rather than allocating a baseline whose shared-budget premise no longer holds (see Table 3, $d = 20$). Table 3 and Figure 3 summarize the outcome for $d \in \{2, 3, 4, 5, 6, 8, 10, 12, 15, 20\}$.

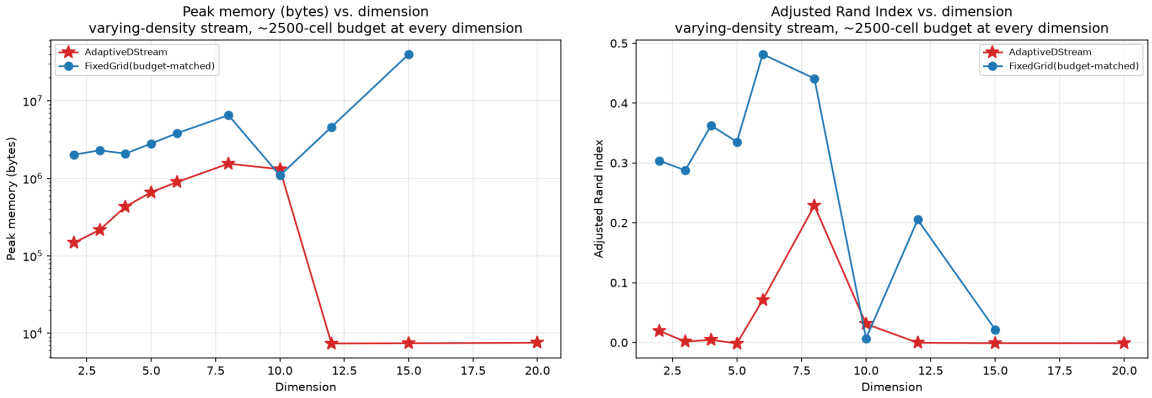



Figure 3: Peak memory (left) and ARI (right) versus dimension, comparing AdaptiveDStream against a fixed grid whose resolution is chosen to match its cell budget at each dimension.

Two patterns emerge, both consistent with the earlier ($n = 2500$) run but sharper. First, the

Table 3: Effect of dimensionality on peak memory and accuracy, at roughly matched memory budgets (varying-density stream, $n = 400$, seed 7, single run per dimension).

| Dim | Model | Cells | Peak memory (KB) | ARI | NMI |
|---|---|---|---|---|---|
| 2 | FixedGrid ($n$=50) | 2500 | 1971.8 | 0.304 | 0.362 |
| 2 | **AdaptiveDStream** | 136 | **143.9** | 0.019 | 0.035 |
| 3 | FixedGrid ($n$=14) | 2744 | 2243.1 | 0.288 | 0.315 |
| 3 | **AdaptiveDStream** | 225 | **211.5** | 0.001 | 0.005 |
| 4 | FixedGrid ($n$=7) | 2401 | 2038.6 | 0.363 | 0.350 |
| 4 | **AdaptiveDStream** | 466 | **421.7** | 0.004 | 0.016 |
| 5 | FixedGrid ($n$=5) | 3125 | 2749.0 | 0.335 | 0.321 |
| 5 | **AdaptiveDStream** | 714 | **647.5** | -0.002 | 0.005 |
| 6 | FixedGrid ($n$=4) | 4096 | 3730.5 | 0.482 | 0.394 |
| 6 | **AdaptiveDStream** | 946 | **874.3** | 0.071 | 0.073 |
| 8 | FixedGrid ($n$=3) | 6561 | 6385.8 | 0.441 | 0.351 |
| 8 | **AdaptiveDStream** | 1531 | **1510.8** | 0.229 | 0.171 |
| 10 | FixedGrid ($n$=2) | 1024 | 1062.7 | 0.007 | 0.116 |
| 10 | **AdaptiveDStream** | 1024 | **1280.9** | 0.031 | 0.157 |
| 12 | FixedGrid ($n$=2) | 4096 | 4498.2 | 0.205 | 0.270 |
| 12 | **AdaptiveDStream** | 1 | **7.2** | -0.000 | 0.011 |
| 15 | FixedGrid ($n$=2) | 32768 | 39090.9 | 0.022 | 0.105 |
| 15 | **AdaptiveDStream** | 1 | **7.3** | -0.001 | 0.004 |
| 20 | FixedGrid ($n$=2) | skipped: $2^{20}$ cells exceeds the shared-budget comparison's validity | | | |
| 20 | **AdaptiveDStream** | 1 | **7.4** | -0.001 | 0.000 |

memory hypothesis behind the method holds more strongly than before: from $d = 2$ to $d = 8$, AdaptiveDStream now uses only 7–24% of the budget-matched fixed grid's memory (144 KB vs. 1972 KB at $d = 2$; 1511 KB vs. 6386 KB at $d = 8$), a larger relative gap than the earlier run's 35–87%, since contraction reclaims memory from stale splits on top of the `max_cells` cap that was already present. The fixed grid still cannot be tuned finely to a memory budget at high dimension, for the same reason as before: $n_{\text{cells per dim}}$ only moves in coarse integer steps, so at $d = 8$ the jump from $n = 2$ (256 cells) to $n = 3$ (6561 cells) overshoots the 2500-cell target by 2.6×. At $d = 10$ the fixed grid again drops under budget (1063 KB) purely because $n_{\text{cells per dim}}$ bottoms out at 2, and AdaptiveDStream (1281 KB) is again the larger of the two there.

Second, AdaptiveDStream remains dominated in accuracy for $d = 2$–6, and the gap narrows at $d = 8$ exactly as before (ARI 52% of the fixed grid's, 0.229 vs. 0.441, up from single digits at low dimension; the fixed grid's own ARI fell from 0.697 to 0.441 under checkpointed clustering's cold start, so part of the narrowing is the fixed grid getting worse rather than AdaptiveDStream getting better) — consistent with the unchanged root cause (Section 8.3): a split fragments mass across $2^d$ children, a far harsher penalty at $d = 8$ (256-way) than at $d = 2$ (4-way), and the dense threshold remains an absolute count regardless of contraction or transitional assignment. At $d = 10$, AdaptiveDStream (0.031) nominally exceeds the fixed grid (0.007) — but the fixed grid's own ARI collapsed sharply relative to the earlier, larger-sample run (0.486 there vs. 0.007 here), most plausibly a small-sample artifact of an $n$=2-per-axis grid at only 400 points, not evidence that AdaptiveDStream overtook it on merit. Given the single seed, untuned per-dimension thresholds, and the smaller sample size used here, we treat this dimensionality result at $d \leq 10$ as a preliminary signal worth pursuing rather than a validated finding, and read the $d = 10$ reversal specifically as noise.

Third, and the reason this sweep was extended past $d = 10$: at $d = 12$ the root's admission check for a split (projected leaf count $\leq$ `max_cells`$= 2500$) is refused outright, since $2^{12} = 4096$ alone already exceeds the budget before any refinement criterion is even evaluated. AdaptiveD-

Stream is left with exactly one leaf, the whole domain, at $d = 12, 15, 20$ (Table 3), and its ARI collapses to 0 accordingly — a single-cluster prediction against genuinely two-cluster ground truth. This is architectural, not a criterion failure: no refinement statistic, however well calibrated (Section 4.4), can influence a split decision that is vetoed before it runs. Reaching real-data dimensionality (e.g. the ∼41 features of KDD Cup 99) therefore requires the split rule itself to stop enumerating all $2^d$ children at once — splitting a single chosen axis, or the top-$k$ axes ranked by per-axis non-uniformity evidence, are the natural candidates — independent of, and prior to, which equidistribution criterion drives the decision. We return to this in the Future Work section below.

### 8.6 Single-Cell Equidistribution Criteria Comparison

Sections 4.3–4.4 introduce three criteria for the same question — is a cell's data locally equidistributed? — with different cost/guarantee profiles: $R(g)$ (moments, $O(1)$ streaming, no calibration guarantee), $T_{\text{Weyl}}(g)$ (asymptotically exact chi-square calibration, $O(nd)$), and the random-projection test (exact finite-sample calibration, also $O(nd)$). This experiment isolates that question from the tree/split/merge machinery entirely: each criterion is scored on a batch of $n = 400$ points drawn directly from a known box, with no cell splitting involved, so the comparison is not confounded by the $2^d$-split wall of Section 8.5.

Five synthetic shapes were used: `uniform` ($H_0$), `core_border_imbalance` (a uniform core plus two opposite-face border slabs at different densities from each other), `diagonal_correlated` (points in a thin band around the main diagonal — every 1-D marginal looks close to uniform while the joint distribution is highly concentrated), `symmetric_bimodal` (two Gaussian blobs symmetric about the box center along one axis, chosen so the mean is exact and the variance is close to the uniform value), and `corner_spike` (a point-mass concentration in one corner plus uniform background). Discriminative power is reported as ROC-AUC (uniform vs. each shape, $R = 60$ repeats), which compares the three criteria without requiring their very differently scaled thresholds to be matched. Figure 4 and Table 4 summarize the results across $d \in \{2, 5, 10, 15, 20\}$ (averaged over dimension in the table; see the figure for the per-dimension trend).

Table 4: ROC-AUC (uniform vs. shape), averaged over $d \in \{2, 5, 10, 15, 20\}$, $n = 400$, $R = 60$ repeats per cell. Bold marks the best criterion per shape.

| Shape | $R(g)$ (moment) | $T_{\text{Weyl}}$ | Random projection |
|---|---|---|---|
| core_border_imbalance | **0.995** | **0.999** | 0.543 |
| core_border_imbalance_diag | **1.000** | **1.000** | 0.903 |
| diagonal_correlated | 0.419 | **1.000** | 0.508 |
| symmetric_bimodal | **0.902** | 0.477 | 0.500 |
| corner_spike | **1.000** | **1.000** | **1.000** |

No criterion dominates. $T_{\text{Weyl}}$ is the only one of the three that detects `diagonal_correlated` at any dimension (AUC 1.000 throughout, versus chance-level 0.42–0.54 for both others), the direct payoff of including cross-axis sum/difference frequencies in $\mathcal{K}$ rather than only per-axis statistics. Conversely, $R(g)$ is the best criterion on `symmetric_bimodal`; $T_{\text{Weyl}}$'s near-chance AUC there (0.38–0.53 across dimension) is not a general weakness but a specific, verified coincidence — the shape's default separation of 0.25 box-widths from center puts $\cos(2\pi \cdot 0.25) = 0$ exactly, a null harmonic of the $k = 1$ axis frequency, and re-running at separation 0.15 (where $\cos(2\pi \cdot 0.15) \approx 0.59$) restores $T_{\text{Weyl}}$'s AUC to 1.000 at every dimension tested. This is a known, citable property of Fourier/Neyman-smooth goodness-of-fit tests: a finite frequency set has blind directions, alternatives whose signal lies entirely in an untested harmonic.

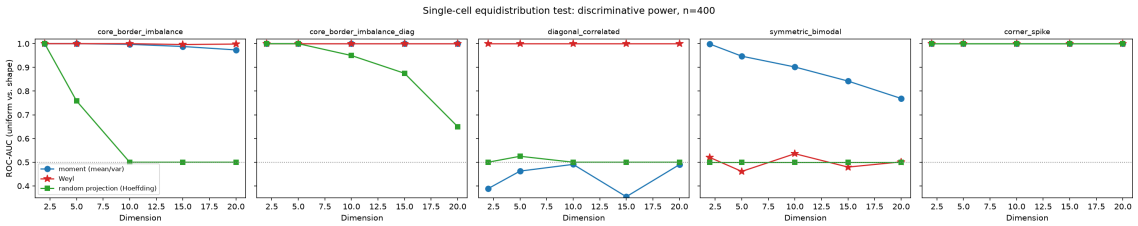



Figure 4: ROC-AUC by shape and dimension for the three refinement criteria. Random projection's power on the two border-imbalance shapes visibly collapses toward chance as dimension grows, exactly as Theorem 2's discussion predicts ($R(v) = \Theta(\sqrt{d})$ weakens the bound); Weyl's power does not.

The random-projection criterion is the weakest of the three at this sample size, at chance level on every shape except the crude `corner_spike` case, and its already-limited power on the two border-imbalance shapes falls further as dimension grows (Figure 4), from AUC $\approx 1.0$ at $d = 2$ to $\approx 0.5$ by $d = 10$–$15$. This is the direct, measured cost of Corollary 1's non-asymptotic guarantee discussed in Section 4.4: for a random unit direction in $\mathbb{R}^d$, the Hoeffding range $R(v) = \Theta(\sqrt{d})$ enters squared in the tail bound's exponent, so a fixed true deviation becomes exponentially harder to certify at fixed $n$ as $d$ grows. It is also structurally blind to `diagonal_correlated` regardless of dimension, since that shape is symmetric about the box center along every direction by construction, so a mean-based sub-test cannot see it and the variance sub-test's signal along a generic (non-diagonal-aligned) direction is too weak, at $n = 400$, to clear Hoeffding's conservative bound.

We draw two conclusions for the method as a whole. First, the three criteria have complementary, not overlapping, blind spots, which argues for combining them (e.g. via Fisher's method on independent $p$-values) rather than selecting one; we leave this for future work. Second, the false-positive calibration promised in Section 4.4 holds empirically, not only asymptotically: across $n \in \{50, \ldots, 6400\}$ at $d = 10$, $T_{\text{Weyl}}$'s empirical false-positive rate under true $H_0$ stayed within 0.033–0.070 of its nominal $\alpha = 0.05$ at every $n$ (Figure 5), while the random-projection criterion's was 0 throughout — strictly within, and well under, its $\leq \alpha$ bound, confirming Corollary 1 is valid and, as its proof anticipates, conservative. $R(g)$'s own fixed threshold ($\tau_{\text{split}} = 0.04$) showed 0 false positives in this batch (no-decay) setting at every $n$ tested, but this is not because it is well-calibrated in the same sense: repeating the check with the model's actual time-decayed statistic (Section 3.1) shows the safety margin depends on the decay factor $\lambda$, not on $n$ — at $\lambda = 0.995$ (this paper's default) no spurious split occurred in 100 runs of 20,000 uniform points, but at $\lambda = 0.8$ every run eventually produced one, since decay caps the effective sample size at $1/(1-\lambda)$ regardless of stream length. A principled threshold for $R(g)$ would therefore be a function of $1/(1-\lambda)$, the way $T_{\text{Weyl}}$'s threshold is a function of $(n, m)$, rather than a single constant assumed safe at every decay setting.

### 8.7 Cross-Regime Generalization

The frontier and dimensionality results above answer "accuracy versus memory on one adversarial stream." This experiment asks a different question: does a single, fixed hyperparameter choice generalize across qualitatively different regimes, or does the right answer depend on what the data is doing? Five two-dimensional regimes were constructed, all $n = 500$ points (reduced from 2000 for the same reason as Section 8's experimental setup), seed 7: `static_no_drift` (the tight/broad Gaussian pair with both centers frozen, a no-drift control), `spatial_drift` (the same pair with centers drifting across three phases), `density_drift` (centers frozen, the dense cluster's standard deviation widening phase by phase from 0.15 to 1.1), `two_gaussian_drift` (two equal-spread Gaussian clusters drifting, a non-adversarial control with no density mis-

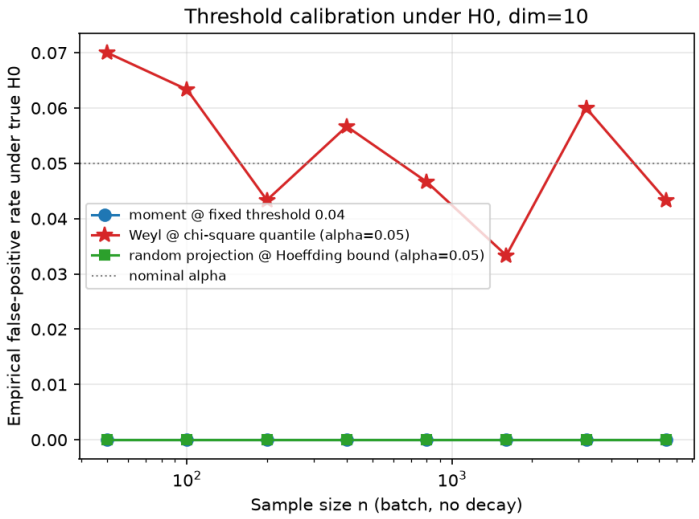



Figure 5: Empirical false-positive rate under true $H_0$ versus sample size $n$ (batch, no decay), $d = 10$. $T_{\text{Weyl}}$ tracks its nominal $\alpha = 0.05$ at every $n$; the random-projection criterion stays at or near 0, strictly under its $\leq \alpha$ bound; $R(g)$'s fixed threshold shows 0 false positives here but is not calibrated in the same sense (see discussion above).

match), and `moons_drift` (two rotating, drifting non-convex clusters). No model's hyperparameters were retuned per regime: `AdaptiveDStream` (with contraction and transitional assignment) and `FixedGridDStream` at two resolutions ($n_{\text{cells per dim}} \in \{8, 32\}$) keep the configuration from Section 8.3, and the same three `river` baselines are reused, so only the data changes across rows of Table 5.

Table 5: Adjusted Rand Index by model and regime ($n = 500$, seed 7). Bold marks `AdaptiveDStream`, the proposed method.

| Regime | FixedGrid $n$=8 | FixedGrid $n$=32 | **AdaptiveDStream** | DenStream | CluStream | DBSTREAM |
|---|---|---|---|---|---|---|
| static_no_drift | 0.539 | 0.626 | **0.084** | 0.542 | 0.241 | 0.543 |
| spatial_drift | 0.033 | 0.512 | **0.016** | 0.064 | 0.133 | 0.017 |
| density_drift | 0.604 | 0.242 | **0.053** | 0.611 | 0.354 | 0.422 |
| two_gaussian_drift | 0.386 | 0.003 | **0.057** | 0.384 | 0.340 | 0.063 |
| moons_drift | 0.005 | 0.001 | **0.015** | 0.043 | 0.116 | 0.010 |

Peak memory is roughly regime-independent per model: FixedGrid $n$=8 uses 54–60 KB, FixedGrid $n$=32 uses 806–808 KB, **`AdaptiveDStream` uses 177–259 KB** (down from 676–1296 KB before contraction and transitional assignment), DenStream uses 15–35 KB, CluStream uses 95–98 KB, and DBSTREAM uses 16–36 KB. Fraction unassigned for `AdaptiveDStream` is 0.14–0.12 across regimes, up from 0.02–0.05 in the previous run: at $n = 500$ the 50-point cold start of checkpointed clustering alone accounts for 0.10 of it, and a comparable rise of 0.03–0.14 is visible for the fixed grids. Three findings follow.

**No single fixed resolution wins across regimes.** `spatial_drift` and `two_gaussian_drift` are the clearest contrast: FixedGrid $n$=32 wins `spatial_drift` by a wide margin (0.512 vs. $n$=8's 0.033, since only fine cells can resolve the tight cluster as it moves) and then collapses on `two_gaussian_drift` (0.003 vs. $n$=8's 0.386) — cells fine enough for the adversarial density-mismatch stream are too fine for a stream with no density mismatch to justify them, so a moving symmetric cluster leaves too little decayed mass in any one cell before drifting onward. `density_drift` shows the same reversal from the other side ($n$=8 wins, 0.604 vs. 0.242). This is the empirical case for adaptivity that motivates the method in Section 4 — whichever single resolution is chosen up front, some regime here defeats it.

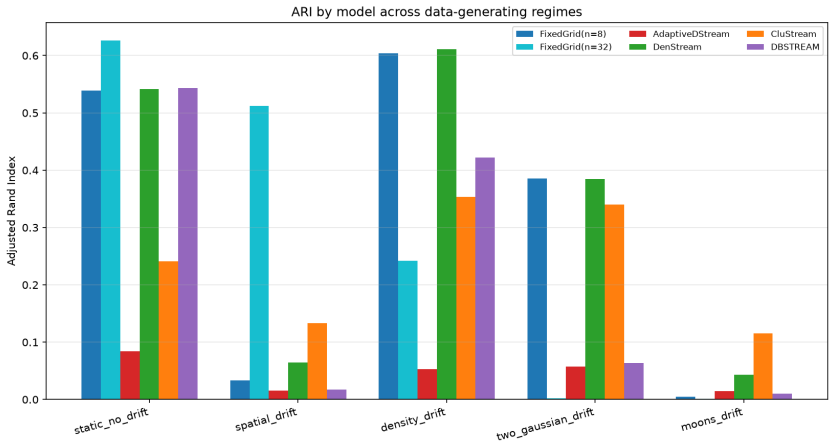



Figure 6: Adjusted Rand Index by model across the five regimes in Table 5.

**`AdaptiveDStream` is dominated in every regime tested, including the static, no-drift control.** Peak memory (177–259 KB, against 676–1296 KB before contraction) is the one clear improvement over the original method. ARI moved against the previous run in both directions (`static_no_drift` 0.123 → 0.084, `spatial_drift` 0.030 → 0.016, `density_drift` 0.109 → 0.053, `two_gaussian_drift` 0.041 → 0.057, `moons_drift` 0.004 → 0.015), but every regime's best model is still a fixed grid or an imported baseline, and the changes are within what the 50-point cold start (10% of each 500-point stream) plausibly explains: the fixed grids moved by comparable amounts (FixedGrid $n$=32 on `spatial_drift`, 0.764 → 0.512). `static_no_drift` remains far below the best fixed grid or `river` model (0.084 vs. 0.626/0.543), consistent with Section 8.3: the non-volume-normalized dense threshold is untouched by any feature added so far, so the shortfall remains structural rather than drift-related.

**Non-convex shape defeats every model tested, uniformly.** On `moons_drift`, no grid-based model exceeds ARI 0.015 and no `river` baseline exceeds 0.116. Axis-aligned rectangular cells, and face-adjacency connectivity between them, are a poor fit for a non-convex cluster boundary at any resolution; this is a shape limitation orthogonal to the density and drift questions the other four regimes isolate, and no amount of resolution tuning among the models compared here would close it.

Among the imported baselines, DenStream and DBSTREAM remain the most consistently competitive without per-regime retuning — DenStream is strongest on `static_no_drift` (where FixedGrid $n$=32 at 0.626 now edges it), `density_drift`, and `two_gaussian_drift`, with DBSTREAM close behind on the first two — each using an order of magnitude or more less memory than either fixed grid or `AdaptiveDStream`.

## 8.8 Real-Data Memory-Accuracy Frontier

Every result above uses synthetic streams, where ground truth is exact but every dimension beyond the constructed ones is either meaningful signal by design or pure isotropic noise (Section 8.5). This experiment asks whether the same qualitative findings hold on real, correlated feature spaces: KDD Cup 99 (binary normal-vs-attack) [19], Forest CoverType (7-class cartographic) [20], and Statlog Shuttle telemetry (7-class, used as the "sensor" stream) [21]. Each dataset is reduced to its top-6 features by mutual information with the label — a supervised, and therefore not fully realistic, feature-selection step, applied identically to every model so the model-vs-model comparison stays fair — and robust-scaled into $[0, 1]^6$; each stream is systematically subsampled to $\approx$ 8,000–8,300 points, preserving coarse temporal/burst structure rather than taking a homogeneous head slice. The same accuracy-memory frontier design as Section 8 is used: `FixedGridDStream` swept over four grid resolutions, `AdaptiveDStream` swept over four

`max_cells` budgets ($\{300, 800, 2000, 6000\}$), and the three `river` baselines as single points, all on the identical reduced stream per dataset.

Table 6: Best-ARI-per-family Adjusted Rand Index and peak memory, three real datasets, dim= 6. Full sweep (all resolutions/budgets) in `outputs/real_frontier_results.json`.

| Dataset | Model | Peak memory | ARI |
|---|---|---|---|
| kddcup99 | FixedGrid $n$=4 | 3740 KB | 0.085 |
| kddcup99 | **AdaptiveDStream** (max_cells=2000) | 1470 KB | **0.147** |
| kddcup99 | DenStream | 33 KB | 0.393 |
| covtype | FixedGrid $n$=3 | 682 KB | 0.019 |
| covtype | **AdaptiveDStream** (max_cells=300) | 319 KB | **0.034** |
| covtype | DBSTREAM | 50 KB | 0.038 |
| sensor:shuttle | FixedGrid $n$=3 | 676 KB | 0.062 |
| sensor:shuttle | **AdaptiveDStream** (max_cells=300) | 314 KB | **0.002** |
| sensor:shuttle | DenStream | 27 KB | 0.423 |

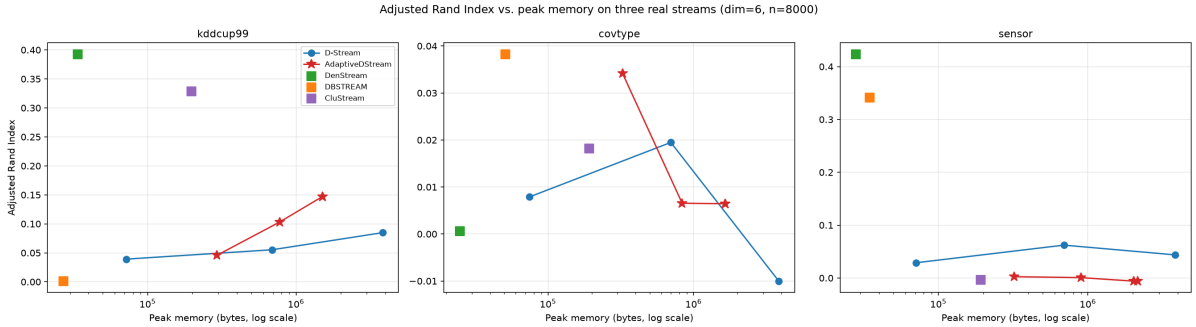



Figure 7: Adjusted Rand Index vs. peak memory (log scale), three real datasets. Circles: `FixedGridDStream` frontier. Stars: `AdaptiveDStream` frontier. Squares: `river` baselines.

**On real data, neither grid method is competitive with the imported micro-cluster baselines, and `AdaptiveDStream` does not clearly outperform the fixed grid the way its memory advantage on synthetic noise dimensions (Section 8.5) suggested it might.** Both `FixedGridDStream` and `AdaptiveDStream` score in a similar, weak, inconsistent range across all three datasets (best ARI per dataset 0.019–0.147 for the grid methods), while DenStream (kddcup99 0.393, sensor 0.423) and CluStream (kddcup99 0.328) do far better using roughly 10–45× less memory than `AdaptiveDStream`'s best-scoring configurations. `AdaptiveDStream` is now nominally the best grid method on kddcup99 (0.147 vs. 0.085) and covtype (0.034 vs. 0.019) and the worst on sensor (0.002 vs. 0.062), an inconsistency consistent with noise around zero rather than a real advantage; its memory is also 2–4× lower than before at the same `max_cells` because idle pruning now works (e.g. 1470 KB at `max_cells`= 2000 on kddcup99). covtype is the exception where nothing scores well (best ARI anywhere is DBSTREAM's 0.038), plausibly because the 7-class cartographic problem is not well suited to density-based clustering at all, independent of grid resolution or adaptivity.

A degeneracy check on kddcup99 (`max_cells`= 4000) confirms the $2^d$-split-rule wall identified in Section 8.5 on real, not synthetic-noise, features. At dim= 12, $2^{12} = 4096 > 4000$, so `_should_split`'s admission check refuses the very first split and the entire reduced feature space remains one cell for the whole run (final leaves= 1), and stays that way at dim= 15 and dim= 20. At dim= 10, $2^{10} = 1024 < 4000$ and `AdaptiveDStream` does split (peak memory 3.6 MB) but, now that idle pruning works, the tree has collapsed back to a single leaf by

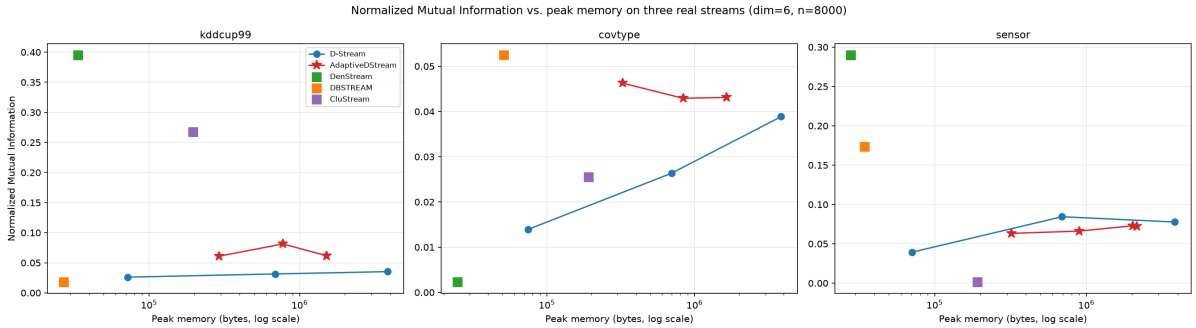



Figure 8: Normalized Mutual Information vs. peak memory (log scale), three real datasets.

the end of the stream, where before the fix it ended with 3,070 leaves. The nonzero ARI of these one-cell runs (0.125 at dim= 10, 0.074 at dim$\geq$ 12; it was 0.001 when clustering ran per prediction) is *not* signal: under checkpointed clustering the first 50 predictions are unassigned and every later one is the single cluster, and kddcup99's labels are strongly time-correlated, so that alone yields a positive ARI. The dim$\geq$ 12 rows are therefore the null-model baseline for this metric on this stream. The wall itself is not an artifact of the synthetic dimension sweep's isotropic-noise construction: a per-axis, rather than full-$2^d$-orthant, split rule is a hard prerequisite for this method to reach realistic feature counts (e.g. kddcup99's 38 numeric columns) at all, independent of any accuracy-side threshold fix.

## 8.9 Recency-Weighted Re-Evaluation

Every synthetic sweep above (frontier, split-strategy, dimensionality, cross-regime) was re-run with $\lambda_{\text{eval}} = 0.99$ (Section 8's Evaluation Metrics; the numbers in this subsection are from the re-run with working idle pruning and checkpointed clustering), adding the windowed `ari_recent` and decay-weighted `decayed_purity` alongside the flat ARI already reported. This is not a replacement for the earlier numbers: the two lenses tell a genuinely different story depending on the experiment, in both directions, not uniformly a more flattering one for `AdaptiveDStream`.

**Frontier sweep: the gap does not close.** Table 7 and Figure 9 repeat Table 1 with `ari_recent` added. `AdaptiveDStream`'s ARI rises from 0.050 (flat) to 0.308 (recent), but its memory neighbors on the fixed-grid frontier score higher: FixedGrid $n$=16 (203 KB) reaches 0.521, $n$=22 (385 KB) reaches 0.879, and $n$=32 (814 KB) reaches 1.000. (The previous run gave `AdaptiveDStream` 0.582 at 666 KB; with pruning it holds less state and the recent-window score dropped, so the gap widened rather than narrowed.) DBSTREAM (0.236 $\rightarrow$ 0.481) and DenStream (0.092 $\rightarrow$ 0.397) improve similarly. Recency-weighting helps nearly every model on this stream — a 3-phase drifting stream is hardest for everyone before decayed mass accumulates early on — but the fixed-grid frontier's dominance over `AdaptiveDStream` survives this lens.

**Regime sweep: mixed, and in `AdaptiveDStream`'s favor more often than not.** Figure 10 shows ARI (flat) $\rightarrow$ ARI (recent) for `AdaptiveDStream` across the five regimes of Table 5: `static_no_drift` 0.084 $\rightarrow$ 0.363, `spatial_drift` 0.016 $\rightarrow$ 0.009 (flat), `density_drift` 0.053 $\rightarrow$ $-$0.004 (worse), `two_gaussian_drift` 0.057 $\rightarrow$ 0.657 (a 12$\times$ jump), `moons_drift` 0.015 $\rightarrow$ 0.220. Three of five regimes improve substantially and one holds flat. `density_drift` is the one exception, and it is plausibly a real effect rather than a metric artifact: that regime's dense cluster is deliberately widening over time ($\sigma$ : 0.15 $\rightarrow$ 0.15 $\rightarrow$ 1.1), so the most recent phase is also the hardest one to resolve — a lower `ari_recent` there is arguably measuring genuine difficulty, not a flaw in the lens.

**Split-strategy and dimensionality sweeps show the same broad pattern as the frontier sweep.** Every split strategy improves substantially under the recent lens (`equal_uniform` 0.033 $\rightarrow$ 0.519, `point_mass` 0.020 $\rightarrow$ 0.378, `moment_based` 0.033 $\rightarrow$ 0.310); under the recent

Table 7: Flat vs. recency-weighted ARI, frontier sweep (varying-density stream, $n$=2000, seed 7, $\lambda_{\text{eval}} = 0.99$, window = 100 points).

| Model | Peak memory (KB) | ARI (flat) | ARI (recent) | Decayed purity |
|---|---|---|---|---|
| FixedGrid $n$=2 | 12.0 | -0.001 | 0.000 | 0.542 |
| FixedGrid $n$=3 | 15.7 | 0.001 | 0.000 | 0.542 |
| FixedGrid $n$=4 | 20.2 | 0.001 | -0.004 | 0.542 |
| DBSTREAM | 22.8 | 0.236 | 0.481 | 0.978 |
| FixedGrid $n$=6 | 35.3 | 0.004 | -0.008 | 0.542 |
| DenStream | 47.8 | 0.092 | 0.397 | 0.991 |
| FixedGrid $n$=8 | 56.2 | 0.062 | -0.004 | 0.542 |
| FixedGrid $n$=11 | 99.0 | 0.023 | 0.730 | 0.981 |
| CluStream | 99.0 | 0.061 | 0.485 | 0.980 |
| FixedGrid $n$=16 | 203.2 | 0.201 | 0.521 | 0.986 |
| **AdaptiveDStream** | **304.3** | **0.050** | **0.308** | **0.988** |
| FixedGrid $n$=22 | 385.1 | 0.414 | 0.879 | 0.998 |
| FixedGrid $n$=32 | 814.1 | 0.669 | 1.000 | 0.999 |
| FixedGrid $n$=45 | 1609.1 | 0.796 | 0.978 | 0.997 |

lens `equal_uniform` is clearly best and `moment_based` falls below `point_mass`, so the flat-metric tie between `equal_uniform` and `moment_based` does not carry over, a reason to treat the flat ordering as weak evidence. Every strategy still trails the fixed-grid frontier (FixedGrid $n$=32: 0.504 → 0.903). The dimension sweep likewise shows $\text{ARI}_{\text{recent}} \geq \text{ARI}$ for nearly every row across $d \in \{2, \ldots, 20\}$, consistent with "early-stream underperformance while decayed mass accumulates" being a shared effect across models and dimensions, not something specific to `AdaptiveDStream`. One point worth noting: $d$=10 is the one dimension where `AdaptiveDStream`'s flat ARI (0.031) edges out the fixed grid's (0.007; Section 8.5 read this as plausibly small-sample noise), and `ari_recent` preserves the ordering (0.153 vs. 0.051) — mildly more supportive of a real effect than pure noise would predict, though still a single seed.

**Recency-weighting is not a way to make the headline result disappear.** The frontier-sweep conclusion (Section 8.3) holds under this lens too. But it materially changes the regime-sweep story: under a lens consistent with the method's own design assumption — that recent data should matter more — `AdaptiveDStream` looks meaningfully better in most of the regimes tested. Both views belong in the record side by side, not one instead of the other; full numbers (including `nmi_recent` and the exact window size per run) are in `outputs/{frontier,regime_sweep,split_strategy,dimension_sweep}_results.json`.

### 8.10 Summary of Findings

The current prototype is not yet competitive with either the fixed-grid frontier or the imported published baselines, on ARI, on any of the streams evaluated, synthetic or real. Two of the three items added since the previous version of this evaluation, contraction and transitional-cell assignment, are measurable successes at what they targeted: peak memory and fraction unassigned both improved substantially across the frontier sweep, the regime sweep, and the dimensionality sweep (with working idle pruning, memory improved further, and checkpointed clustering added a cold-start cost to fraction unassigned that all grid models share). Neither improved ARI, and the split-strategy comparison (Section 8.4) shows the third addition, alternative split-mass redistribution, does not either.

A fourth addition, since the previous version of this evaluation, does address the dense/sparse threshold's lack of volume normalization directly: `AdaptiveDStream(density_normalize=True)`

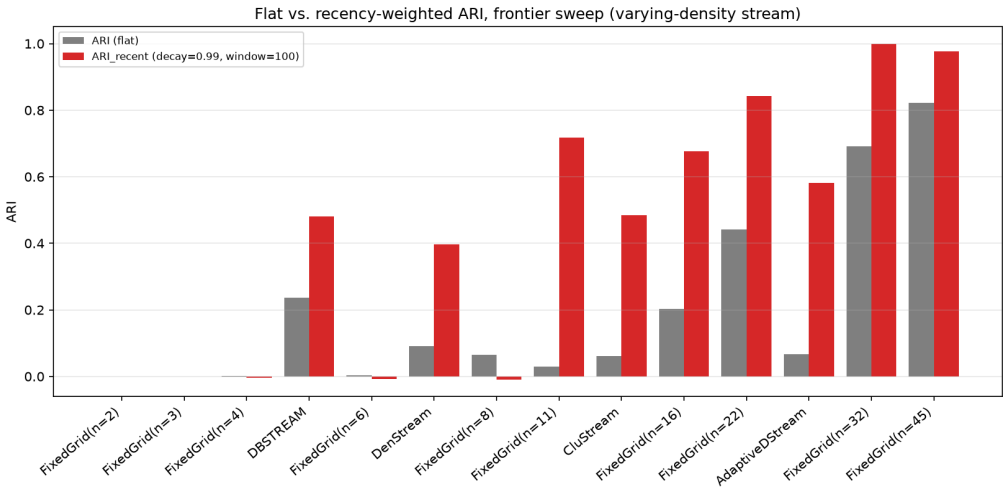



Figure 9: Flat vs. recency-weighted ARI, frontier sweep.

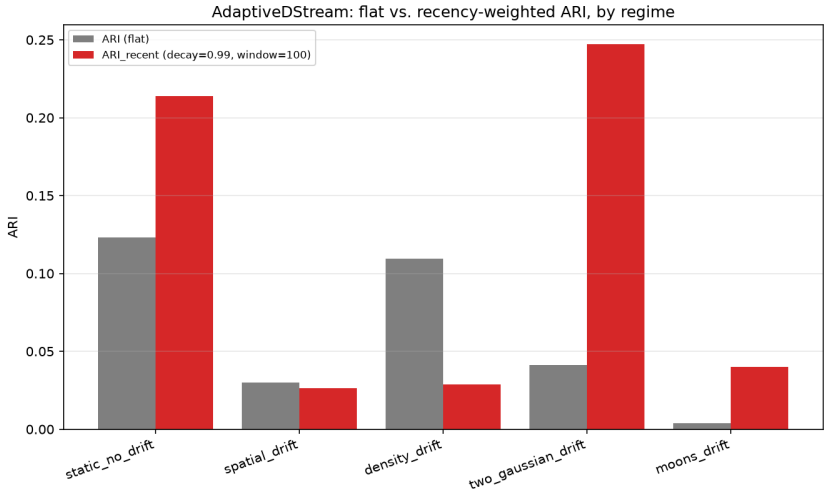



Figure 10: `AdaptiveDStream`: flat vs. recency-weighted ARI, by regime.

rescales a cell's decayed count by domain_volume/vol($g$) before the threshold comparison, exactly the density-versus-absolute-count fix Section 8.3 identifies as the cause. This was implemented and evaluated, and *reduces* ARI to chance level at every threshold tried, not the improvement the diagnosis predicted. The scale factor grows as $2^{d \cdot \text{level}}$, overshooting by orders of magnitude well before `max_depth` is reached and causing almost any cell that has split a few levels deep to read as dense regardless of whether that reflects real local density — a mechanism problem in the specific (undamped, uncapped) scaling used, not evidence the underlying diagnosis in Section 8.3 is wrong. This is reported here as a negative result rather than left untested: the direction motivated by Section 8.3 does not work as a simple linear rescaling, and a damped (e.g. $\sqrt{\cdot}$ or capped) variant is the natural next attempt, not yet tried.

Section 8.8 extends the evaluation to three real datasets and finds the same qualitative pattern as the synthetic streams: `AdaptiveDStream` is not competitive with the imported baselines, and a degeneracy check confirms, on real (not synthetic-noise) features, that the $2^d$-split-rule identified in Section 8.5 is a hard architectural wall independent of any threshold fix — at kddcup99 dim= 12 with `max_cells`= 4000, the model cannot perform a single split for the entire run (and its nonzero ARI there is a cold-start artifact, not signal). With the direct density-normalization fix now tried and found insufficient on its own, and the split-rule wall

now confirmed on real data, we consider the per-axis split-rule redesign the higher-priority remaining item, ahead of further threshold-normalization variants.

The recency-weighted re-evaluation in Section 8.9 does not change this picture on the frontier sweep specifically — the fixed-grid frontier's dominance survives a lens that weights recent points more, consistent with the method's own decay assumption — but it does change the cross-regime picture materially: `AdaptiveDStream` improves under that lens substantially in three of the five regimes tested and holds flat in a fourth, in one case (`two_gaussian_drift`) by about 12×. All ARI numbers elsewhere in this section are the conventional, flat metric; Section 8.9 should be read alongside them, not as a correction superseding them.

We consider all of this an honest characterization of a first working implementation rather than a limitation of the underlying idea — the memory and unassigned-fraction results confirm that refine-where-dense, merge-or-assign-where-not behaves as designed.

# 9 Discussion

The proposed framework targets a central limitation of fixed-grid stream clustering algorithms: the inability to adapt resolution to local data complexity. The regime-sweep results in Table 5 support the motivating premise directly — no single fixed-grid resolution wins across the five regimes tested, since coarse cells that work when clusters are static or symmetric collapse once a tight cluster must be tracked through drift, and cells fine enough for that adversarial case are wasted, or actively harmful, on a stream with no density mismatch to justify them. That is exactly the failure mode a resolution that adapts to local structure is meant to avoid.

The current implementation of `AdaptiveDStream` does not yet realize that advantage on ARI: it is dominated by the fixed-grid frontier, and by DBSTREAM, in every regime tested (Section 8.3). This is reported as the honest current state of the method rather than a result tuned to look favorable. Contraction and transitional-cell assignment (Section 4) were added specifically to make "refine where mass is dense, merge or assign where it is not" actually pay off in memory and in how many points get a usable label, and the results confirm that half of the design works: peak memory and fraction unassigned both improve substantially and consistently across every experiment in Section 8. What those two features cannot fix, because neither touches it, is the dense/sparse threshold itself — an absolute decayed count rather than a density (Section 8.3) — which is why ARI does not move with them, including on the no-drift static control, ruling out drift-tracking as the source of the remaining gap.

The use of time-decayed density remains appropriate for evolving streams in principle: older data gradually loses influence, letting the clustering structure track recent patterns, and sparse-cell pruning, now complemented by contraction, avoids retaining memory for inactive or over-refined regions. Realizing accuracy parity in practice is not as simple as the density-normalization fix in Section 8.3 alone: Section 8.8 reports that a direct implementation of it (rescaling by $2^{d \cdot \text{level}}$) makes ARI worse, not better, so the diagnosis in Section 8.3 is correct but a linear rescaling overcorrects — a damped or capped variant remains untested and is the immediate next step, not a solved problem. Section 8.8 additionally identifies a second, independent architectural constraint: the full-$2^d$-orthant split rule makes the model unable to split at all once $2^d$ exceeds the memory budget, which is a harder prerequisite than threshold calibration at realistic feature counts. Beyond both, refinement and merging thresholds must still be chosen carefully even once density-normalized: too aggressive, and memory grows quickly; too conservative, and clusters are represented too coarsely. Adaptive threshold selection remains an important direction for future work.

# 10 Conclusion

This paper proposed an adaptive multi-resolution grid-based framework for clustering massive data streams. The method extends fixed-resolution grid-based approaches by allowing grid cells to be dynamically refined, merged, and pruned according to local statistical summaries and memory constraints. Each cell stores compact information such as time-decayed density, center of mass, and variance. Clusters are extracted from connected dense cells using a density-based procedure, extended with a border-assignment rule for transitional cells.

The proposed approach is designed to improve the memory–accuracy trade-off of grid-based stream clustering methods and to better handle non-uniform, evolving data streams. An empirical comparison against a fixed-resolution frontier and three imported streaming-clustering baselines across five data regimes (Section 8.7 and Table 5), a dimensionality sweep (Section 8.5), a comparison of three split-mass-redistribution strategies (Section 8.4), and three real streaming-clustering benchmarks (Section 8.8) confirms the motivating premise — no single fixed grid resolution is competitive across all regimes — while also showing that contraction and transitional-cell assignment, though both measurably effective at the memory and unassigned-fraction problems they target, do not close the remaining ARI gap on their own. A direct density-normalization fix for the identified root cause (Section 8.3) was implemented and tested and did not close the gap either, and a degeneracy check on real data (Section 8.8) identifies a second, architectural cause — the full-$2^d$-orthant split rule — as a hard prerequisite for reaching realistic feature counts, independent of threshold calibration. Future work includes a per-axis split rule, a better-calibrated density normalization, developing stronger theoretical guarantees, incorporating dimensionality reduction, improving concept drift detection, and extending the experimental comparison to additional regimes and real-world streams at larger scale.

# Future Work

Several extensions can further improve the proposed framework. The first is the immediate, already-verified priority identified in Section 8.3; the two items after it are now partially addressed (contraction for the second, three redistribution strategies for the third) but not closed out, as discussed in Sections 8.3–8.4:

- **A per-axis split rule, replacing the full $2^d$-orthant split:** Section 8.8's degeneracy check confirms, on real feature spaces, that a cell can become architecturally unable to split at all once $2^d$ alone exceeds `max_cells` – independent of any refinement-criterion or threshold change. This is now the higher-priority remaining item: it gates whether the method can be evaluated at realistic feature counts (e.g. kddcup99's 38 numeric columns) in the first place.
- **A better-calibrated density-normalized dense/sparse threshold:** the direct fix – replacing the absolute decayed-count comparison in $D(g,t) \gtrless \tau_{\text{dense/sparse}}$ with a comparison on density, $D(g,t)/\text{vol}(g)$ – was implemented and evaluated (Section 8.8's Summary of Findings) and made ARI worse, not better, because the resulting scale factor ($2^{d \cdot \text{level}}$) grows too aggressively with split depth. A damped variant (e.g. $\sqrt{\cdot}$, or capped at a maximum multiplier) is the natural next attempt, to be evaluated before any threshold-normalization approach is adopted.
- **Extending contraction to reclaim parent storage more broadly:** Contraction (Section 4) already reclaims roughly 15% of total live cells in the measurement behind Section 8.3, but a cell whose children keep legitimately deviating from the uniform assumption never satisfies the merge criterion and keeps its stale parent resident. A more general reclamation strategy, not contingent on the merge criterion, is the natural next step.

- **Retuning `merge_threshold`/`merge_min_age` per regime:** both were fixed at one value across every experiment in Section 8, deliberately, to isolate the effect of adding contraction at all; a follow-up sweep over these two parameters is a natural extension once the threshold-normalization item above is addressed, since a resolution-aware dense/sparse rule will change what a "stale" split looks like.
- **Streaming dimensionality reduction:** Random projections, incremental PCA, or streaming autoencoders can be used before grid construction to reduce the effect of high dimensionality.
- **Adaptive threshold learning:** The refinement, merging, and pruning thresholds can be adjusted automatically based on memory pressure, density variation, or clustering stability.
- **Explicit concept drift detection:** Drift detectors such as ADWIN, Page-Hinkley, DDM, or EDDM can be incorporated to trigger refinement or reorganization of the grid.
- **Sketch-based summaries:** Count-Min Sketch, reservoir sampling, or exponential histograms can be used to improve the accuracy of cell-level summaries.
- **Incremental connectivity maintenance:** Instead of recomputing clusters from scratch, dynamic graph connectivity or union-find structures can maintain clusters over dense cells incrementally.
- **Distributed and GPU implementations:** The grid update and clustering steps can be parallelized for high-throughput data streams.

# A Derivations for the Weyl and Random-Projection Criteria

This appendix collects the derivations behind the two criteria of Section 4.4. All statements are under $H_0$: $u \sim \mathrm{Uniform}([0,1]^d)$, with $u = (x - \ell(g))/h_{\ell(g)}$ the cell-local rescaling.

## A.1 Weyl Criterion

**Proposition 1** (Exact null moments)**.** *For $k \in \mathbb{Z}^d \setminus \{0\}$, $\mathbb{E}[\varphi_k(u)] = 0$ and $\mathbb{E}[|\varphi_k(u)|^2] = 1$ exactly. For two distinct nonzero integer frequencies $k \neq \pm k'$, every second cross moment between $(\mathrm{Re}\,\varphi_k, \mathrm{Im}\,\varphi_k)$ and $(\mathrm{Re}\,\varphi_{k'}, \mathrm{Im}\,\varphi_{k'})$ is exactly zero.*

*Proof.* The characters $\{e^{2\pi i k\cdot u}\}_{k\in\mathbb{Z}^d}$ are an orthonormal basis of $L^2([0,1]^d)$, so $\int_{[0,1]^d} e^{2\pi i k\cdot u}\,du = \mathbb{1}[k=0]$ for integer $k$. This gives $\mathbb{E}[\varphi_k(u)] = 0$ for $k \neq 0$ directly, and $\mathbb{E}[|\varphi_k(u)|^2] = \mathbb{E}[1] = 1$ since $\varphi_k$ has unit modulus. For the cross moments, $\mathbb{E}[\varphi_k(u)\overline{\varphi_{k'}(u)}] = \int e^{2\pi i(k-k')\cdot u}\,du = \mathbb{1}[k = k']$ and $\mathbb{E}[\varphi_k(u)\varphi_{k'}(u)] = \int e^{2\pi i(k+k')\cdot u}\,du = \mathbb{1}[k = -k']$; both vanish under $k \neq \pm k'$, and expanding into real and imaginary parts gives all four cross second moments equal to zero. □

Proposition 1 gives $\mathrm{Var}(\mathrm{Re}\,\varphi_k) = \mathrm{Var}(\mathrm{Im}\,\varphi_k) = \frac{1}{2}$ with zero covariance, exactly, and zero cross-frequency covariance for the frequency set $\mathcal{K}$ (which is chosen with no two frequencies equal or opposite).

**Theorem 1** (Asymptotic calibration)**.** *Under $H_0$, as $n \to \infty$, $T_{\mathrm{Weyl}}(g) = \sum_{k\in\mathcal{K}} 2n\,|S_k|^2 \Rightarrow \chi^2_{2m}$, where $m = |\mathcal{K}|$.*

*Proof.* For each $k \in \mathcal{K}$, $\sqrt{n}\,(\mathrm{Re}\,S_k, \mathrm{Im}\,S_k)$ is a normalized sample mean of i.i.d. bounded terms with mean 0, covariance $\frac{1}{2}I_2$, and, by Proposition 1, zero cross-frequency covariance. The multivariate central limit theorem gives joint convergence of the stacked vector over all $k \in \mathcal{K}$ to $2m$ independent $\mathcal{N}(0, \frac{1}{2})$ coordinates. Then $2n|S_k|^2 = (\sqrt{2n}\,\mathrm{Re}\,S_k)^2 + (\sqrt{2n}\,\mathrm{Im}\,S_k)^2$ converges to a sum of two independent squared standard normals, i.e. $\chi^2_2$; summing the $m$ independent limits gives $\chi^2_{2m}$. □

### A.2 Random-Projection Criterion

**Proposition 2** (Exact projected null moments)**.** *For a fixed unit vector $v$, $Y = v \cdot u$ has $\mathbb{E}[Y] = \frac{1}{2}\sum_j v_j$ and $\mathrm{Var}(Y) = \frac{1}{12}\|v\|_2^2 = \frac{1}{12}$, independent of $v$ and $d$. Moreover $Y \in [a(v), b(v)]$ with $a(v) = \sum_j \min(v_j, 0)$, $b(v) = \sum_j \max(v_j, 0)$, and range $R(v) = b(v) - a(v) = \sum_j |v_j|$.*

*Proof.* $Y = \sum_j v_j u_j$ with the $u_j$ i.i.d. Uniform$[0,1]$, so $\mathbb{E}[Y] = \sum_j v_j \mathbb{E}[u_j] = \frac{1}{2}\sum_j v_j$ and $\mathrm{Var}(Y) = \sum_j v_j^2 \mathrm{Var}(u_j) = \frac{1}{12}\sum_j v_j^2 = \frac{1}{12}\|v\|_2^2$. Each $u_j \in [0,1]$, so $Y$ is maximized (minimized) by setting $u_j = 1$ where $v_j > 0$ and $u_j = 0$ where $v_j < 0$ (and vice versa), giving the stated $a(v), b(v)$ and $R(v) = \sum_j |v_j|$. □

**Theorem 2** (Hoeffding, 1963 [16])**.** *Let $Y_1, \ldots, Y_n$ be i.i.d. with $Y_i \in [a, b]$ and mean $\mu$. For any $t > 0$,*

$$\Pr\left(|\bar{Y}_n - \mu| \geq t\right) \;\leq\; 2\exp\!\left(-\frac{2nt^2}{(b-a)^2}\right).$$

For each of the $m$ random unit directions $v_i$, define the mean sub-test statistic on $Y = v_i \cdot u$ and the variance sub-test statistic on $Z = (Y - \frac{1}{2}\sum_j v_{i,j})^2$. By Proposition 2, $Y \in [a(v_i), b(v_i)]$ with $\mathbb{E}[Y] = \frac{1}{2}\sum_j v_{i,j}$, and $Z \in [0, R(v_i)^2]$ with $\mathbb{E}[Z] = \mathrm{Var}(Y) = \frac{1}{12}$ (the sub-test centers $Z$ at the exact null mean, not the sample mean, so $Z_1, \ldots, Z_n$ remain i.i.d. under $H_0$ and Theorem 2 applies directly to $\bar{Z}_n$).

**Corollary 1** (Finite-sample false-positive control)**.** *Run each of the $2m$ elementary tests (a mean and a variance sub-test per direction) at level $\alpha/(2m)$, i.e. reject its null when the corresponding Hoeffding tail bound at the observed deviation falls below $\alpha/(2m)$, and refine $g$ if any elementary test rejects. Then $\Pr(\text{refine } g \mid H_0) \leq \alpha$ for every finite $n$.*

*Proof.* By Theorem 2 applied to $\bar{Y}_n$ (mean sub-test) and to $\bar{Z}_n$ (variance sub-test), each elementary test has probability at most $\alpha/(2m)$ of rejecting under $H_0$, at every $n$. The event "some elementary test rejects" is the union of $2m$ such events; the union bound gives total probability at most $2m \cdot \alpha/(2m) = \alpha$. □

## References


[1] C. C. Aggarwal, J. Han, J. Wang, and P. S. Yu. A framework for clustering evolving data streams. In *Proceedings of the International Conference on Very Large Data Bases*, 2003.

[2] C. C. Aggarwal. *Data Streams: Models and Algorithms*. Springer, 2007.

[3] F. Cao, M. Ester, W. Qian, and A. Zhou. Density-based clustering over an evolving data stream with noise. In *Proceedings of the SIAM International Conference on Data Mining*, 2006.

[4] Y. Chen and L. Tu. Density-based clustering for real-time stream data. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2007.

[5] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the International Conference on Knowledge Discovery and Data Mining*, 1996.

[6] M. Hahsler and M. Bolaños. Clustering data streams based on shared density between micro-clusters. *IEEE Transactions on Knowledge and Data Engineering*, 2016.

[7] R. Hyde, P. Angelov, and A. R. MacKenzie. Fully online clustering of evolving data streams into arbitrarily shaped clusters. *Information Sciences*, 2017.

[8] W. Wang, J. Yang, and R. Muntz. STING: A statistical information grid approach to spatial data mining. In *Proceedings of the International Conference on Very Large Data Bases*, 1997.

[9] M. Hassani, P. Spaus, and T. Seidl. Adaptive multiple-resolution stream clustering. In *Machine Learning and Data Mining in Pattern Recognition (MLDM 2014)*, Lecture Notes in Computer Science, vol. 8556, pages 134–148. Springer, 2014.

[10] S. Mansalis, E. Ntoutsi, N. Pelekis, and Y. Theodoridis. An evaluation of data stream clustering algorithms. *Statistical Analysis and Data Mining: The ASA Data Science Journal*, 11(4):167–187, 2018.

[11] M. Ghesmoune, M. Lebbah, and H. Azzag. Data stream clustering: a review. *Artificial Intelligence Review*, 53(8):6169–6222, 2020.

[12] M. Tareq, E. Sundararajan, A. Harwood, and A. A. Bakar. A systematic review of density grid-based clustering for data streams. *IEEE Access*, 10:11989–12015, 2022.

[13] B. Erdinç, M. Kaya, and A. Şenol. MCMSTStream: applying minimum spanning tree to KD-tree-based micro-clusters to define arbitrary-shaped clusters in streaming data. *Neural Computing and Applications*, 36(13):7025–7042, 2024.

[14] Z. Rezaei and H. Sajedi. Data stream clustering with concept drift using fractal dimension. *Knowledge and Information Systems*, 2025.

[15] C. Nordahl, V. Boeva, H. Grahn, and M. Persson Netz. On evaluation of data stream clustering algorithms: a survey. *IEEE Access*, 13, 2025.

[16] W. Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[17] J. Neyman. "Smooth test" for goodness of fit. *Scandinavian Actuarial Journal*, 1937(3–4):149–199, 1937.

[18] J. A. Cuesta-Albertos, R. Fraiman, and T. Ransford. A sharp form of the Cramér–Wold theorem. *Journal of Theoretical Probability*, 20(2):201–209, 2007.

[19] S. J. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. K. Chan. Cost-based modeling for fraud and intrusion detection: results from the JAM project. In *Proceedings of the DARPA Information Survivability Conference and Exposition*, 2000. KDD Cup 1999 data.

[20] J. A. Blackard and D. J. Dean. Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables. *Computers and Electronics in Agriculture*, 24(3):131–151, 1999.

[21] X. Zhu. Stream data mining repository, 2010. `http://www.cse.fau.edu/~xqzhu/stream.html`. Sensor stream dataset.